

Mở rộng hệ thống và tối ưu hiệu suất

MỞ RỘNG HỆ THỐNG VÀ TỐI ƯU HIỆU SUẤT

AGENDA

- 01 Cân bằng tải (Load Balancer)
- 02 Bộ nhớ đệm (Caching)
- 03 Hàng đợi thông điệp (Message Queue)
- 04 Bài tập thực hành

Cân bằng tải (Load Balancer)

Load Balancer là gì?

Load Balancer là thành phần đứng trước một cụm máy chủ, phân phối lưu lượng đến các máy chủ phía sau để không máy nào bị quá tải.

- ☕ Đóng vai trò như *nhân viên điều phối* tại quầy giao dịch ngân hàng
- ☕ Mỗi request từ người dùng được hướng đến một máy chủ còn trống
- ☕ Người dùng chỉ biết một **địa chỉ duy nhất** — không biết số máy chủ phía sau
- ☕ Toàn bộ cụm máy chủ hoạt động như ***một máy chủ ảo duy nhất***

Vai trò của Load Balancer

Load Balancer giúp hệ thống **chịu tải tốt hơn, ổn định hơn** và **dễ mở rộng hơn**.

- ☕ **Phân phối đều tải** — tránh một máy gánh hết trong khi các máy khác rảnh rỗi
- ☕ **Tăng khả năng mở rộng** — thêm máy chủ mới, Load Balancer tự phân phối thêm
- ☕ **Tăng độ sẵn sàng** — nếu một máy chủ lỗi, lưu lượng được chuyển sang máy khác
- ☕ **Đơn giản hóa kiến trúc** — người dùng kết nối qua một URL/IP duy nhất

Các thuật toán phân phối (1/2)

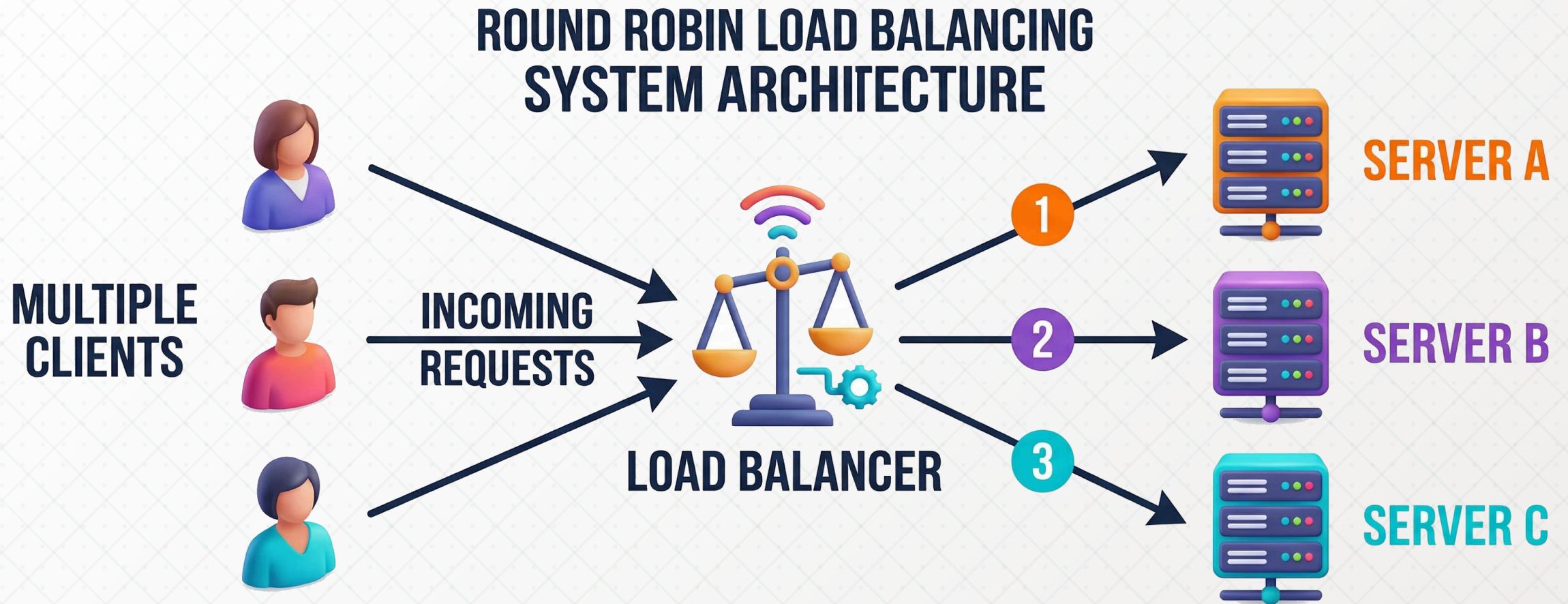
Load Balancer dùng **thuật toán phân phối** để quyết định máy chủ nào nhận request tiếp theo.

- ☕ **Round Robin** — chia lượt theo vòng tròn: $A \rightarrow B \rightarrow C \rightarrow A \rightarrow \dots$, đơn giản và đều
- ☕ **Weighted Round Robin** — như Round Robin nhưng máy mạnh nhận nhiều request hơn theo *trọng số*
- ☕ **Least Connections** — gửi request đến máy chủ đang xử lý **ít kết nối nhất**

Các thuật toán phân phối (2/2)

- ☕ **IP Hash** — dùng địa chỉ IP của client để gán cố định vào một máy chủ cụ thể
- ☕ **Least Response Time** — ưu tiên máy chủ phản hồi *nhANH NHẤT* tại thời điểm đó
- ☕ **Geo-location** — phân phối theo *vị trí địa lý*, gần máy chủ nào thì vào máy đó

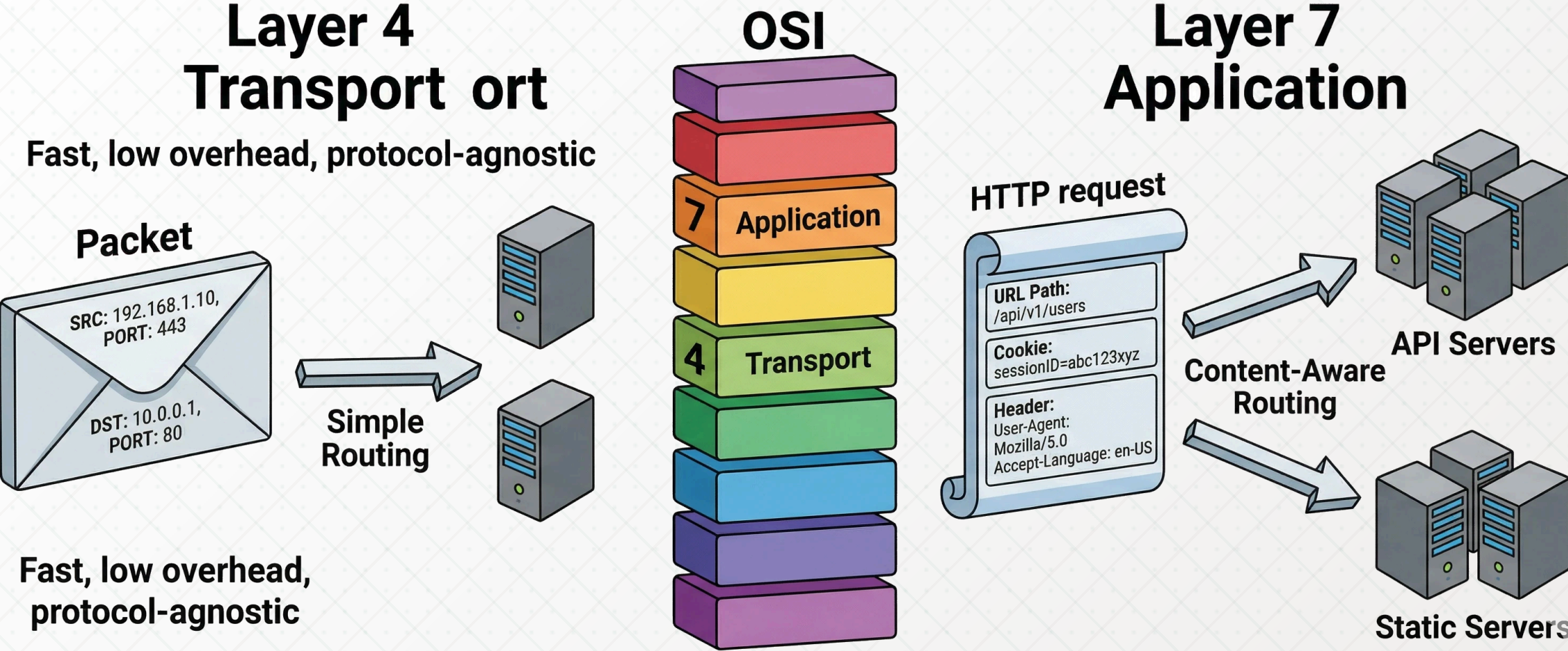
IP Hash hữu ích khi cần duy trì **session stickiness** — cùng một người dùng luôn vào cùng một máy chủ.



Load Balancer tầng 4 vs tầng 7

Phân loại dựa trên **mô hình OSI**: tầng 4 hoạt động ở mức mạng, tầng 7 hiểu nội dung ứng dụng.

- ☕ **Layer 4 (Transport)** — chỉ thấy địa chỉ IP và port của gói tin TCP/UDP
- ☕ **Layer 7 (Application)** — đọc được URL, HTTP header, cookie, và nội dung body
- ☕ Layer 4: nhanh hơn, ít tốn tài nguyên, không cần hiểu giao thức ứng dụng
- ☕ Layer 7: thông minh hơn, hỗ trợ *content-based routing* và terminate SSL



Khi nào dùng Layer 4, khi nào dùng Layer 7?

Chọn theo **nhu cầu thực tế**: Layer 4 cho tốc độ tối đa, Layer 7 cho điều phối thông minh.

Layer 4 — phù hợp khi:

- ☕ Chỉ cần phân tải đơn thuần
- ☕ Giao thức không phải HTTP (UDP, VPN, game server)
- ☕ Yêu cầu độ trễ cực thấp
- ☕ Không cần routing theo nội dung

Layer 7 — phù hợp khi:

- ☕ Web app cần routing theo URL hoặc cookie
- ☕ Cần tách traffic tĩnh/động hoặc mobile/PC
- ☕ Cần terminate SSL tại Load Balancer
- ☕ Hầu hết website và API lớn dùng Layer 7

Khi nào cần sử dụng Load Balancer?

Bất kỳ hệ thống nào cần **mở rộng quy mô** hoặc **nâng cao tính ổn định** đều sẽ cần Load Balancer.

- ☕ **Lượng người dùng tăng** — hàng ngàn/triệu lượt truy cập, một máy chủ không đủ
- ☕ **High Availability** — muốn hệ thống hoạt động liên tục dù có máy chủ bị lỗi
- ☕ **Tách biệt chức năng** — phân loại traffic theo URL để gửi đến đúng nhóm server
- ☕ **Hiệu năng và bảo mật** — tích hợp nén nội dung, cache tạm, tường lửa ứng dụng (WAF)

Bộ nhớ đệm (Caching)

Cache là gì?

Cache là cơ chế lưu trữ tạm thời dữ liệu để phục vụ các yêu cầu truy xuất **nhANH hơn** mà không cần đến nguồn gốc mỗi lần.

- ☕ Dữ liệu được lưu ở nơi truy xuất nhanh — thường là **RAM**
- ☕ Tương tự việc **nhớ đường** sau lần đầu đi: lần sau không cần mở bản đồ
- ☕ **Cache hit** — tìm thấy dữ liệu trong cache, trả về ngay lập tức
- ☕ **Cache miss** — không có trong cache, lấy từ nguồn gốc rồi lưu lại vào cache

Lợi ích của caching

Caching *tăng tốc độ phản hồi, giảm tải backend*, và *tiết kiệm chi phí hạ tầng*.

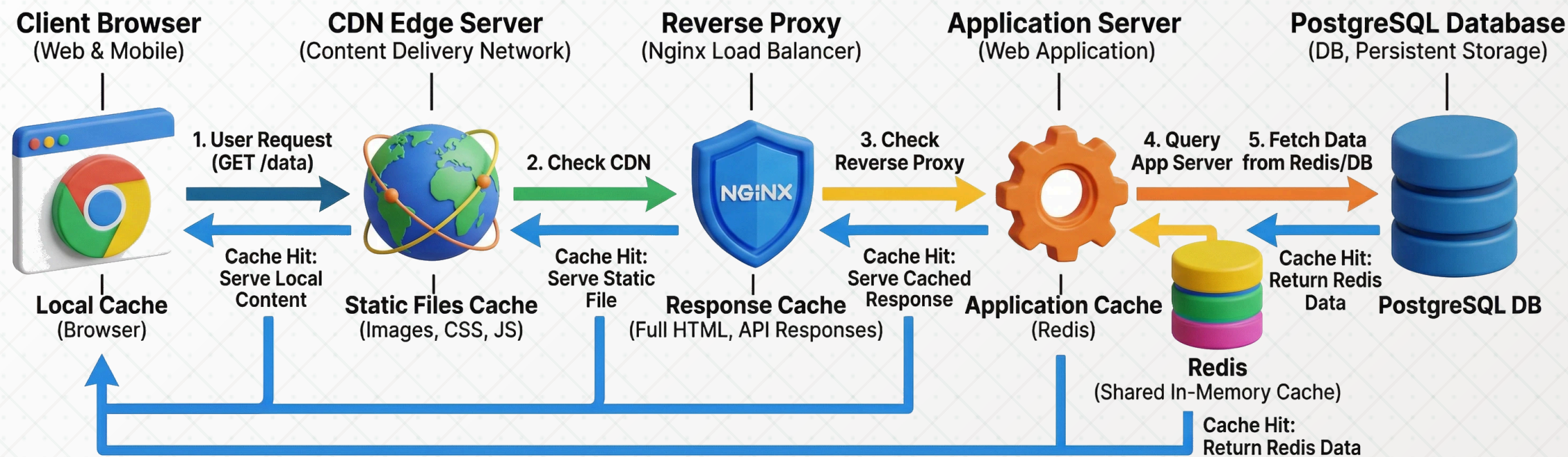
- ☕ **Tăng tốc độ** — cache trong RAM nhanh hơn đọc từ database hàng trăm lần
- ☕ **Giảm tải database** — ít truy vấn đến database gốc hơn, phục vụ được nhiều người dùng hơn
- ☕ **Tiết kiệm chi phí** — thêm lớp cache (Redis) thay vì mua thêm database server mạnh hơn
- ☕ **Trải nghiệm tốt hơn** — thời gian phản hồi nhanh, ứng dụng mượt mà cho người dùng

Các vị trí đặt cache

Mục tiêu chung: đưa dữ liệu **càng gần người dùng càng tốt**, trong môi trường **truy xuất nhanh nhất** có thể.

- ☕ **Client cache** — trình duyệt lưu file tĩnh (CSS, JS, ảnh); điện thoại lưu dữ liệu offline
- ☕ **CDN cache** — máy chủ biên phân phối nội dung tĩnh từ vị trí địa lý gần người dùng
- ☕ **Reverse Proxy cache** — Nginx/Varnish cache kết quả response trước khi đến app server
- ☕ **Application cache** — lưu kết quả truy vấn tốn kém vào Redis/Memcached trong ứng dụng
- ☕ **Database cache** — buffer pool trong DBMS hoặc Redis như lớp đọc nhanh trước database

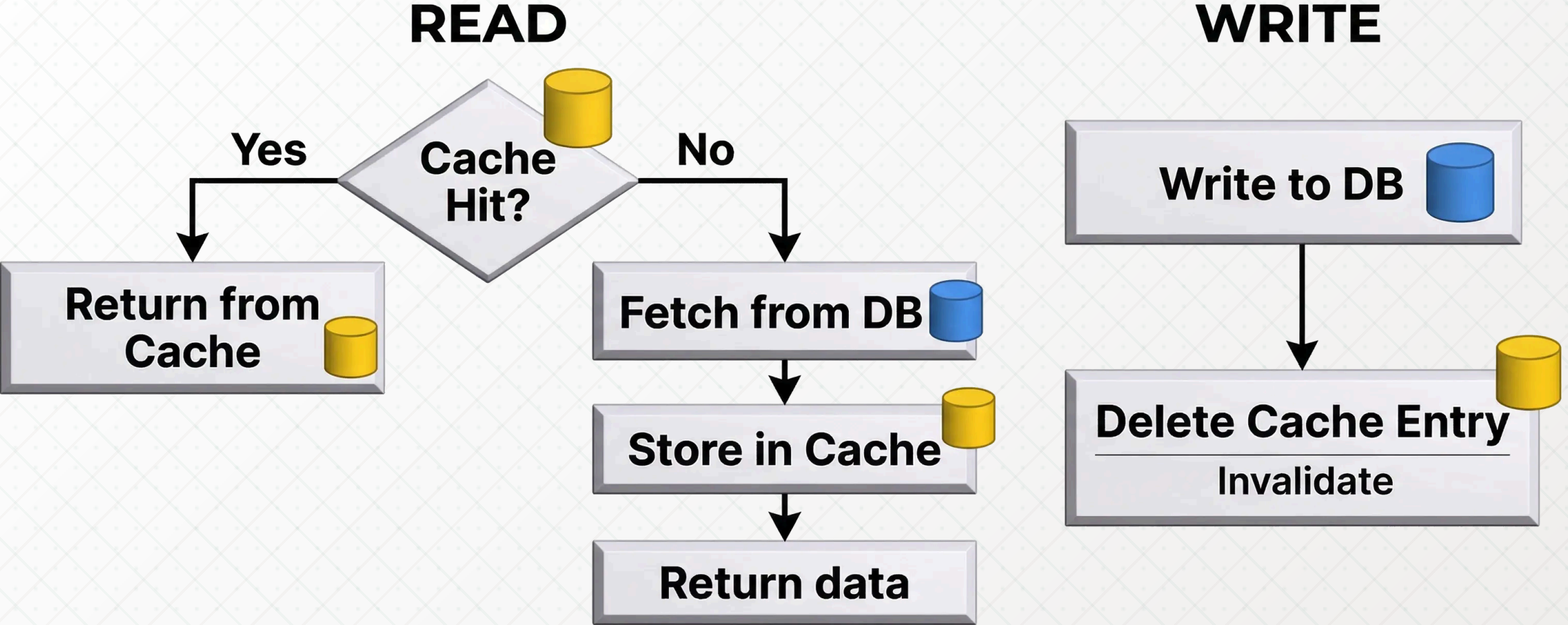
SYSTEM CACHING ARCHITECTURE



Cache-Aside (Lazy Loading)

Chỉ nạp dữ liệu vào cache **khi cần** — đây là chiến lược phổ biến và dễ triển khai nhất.

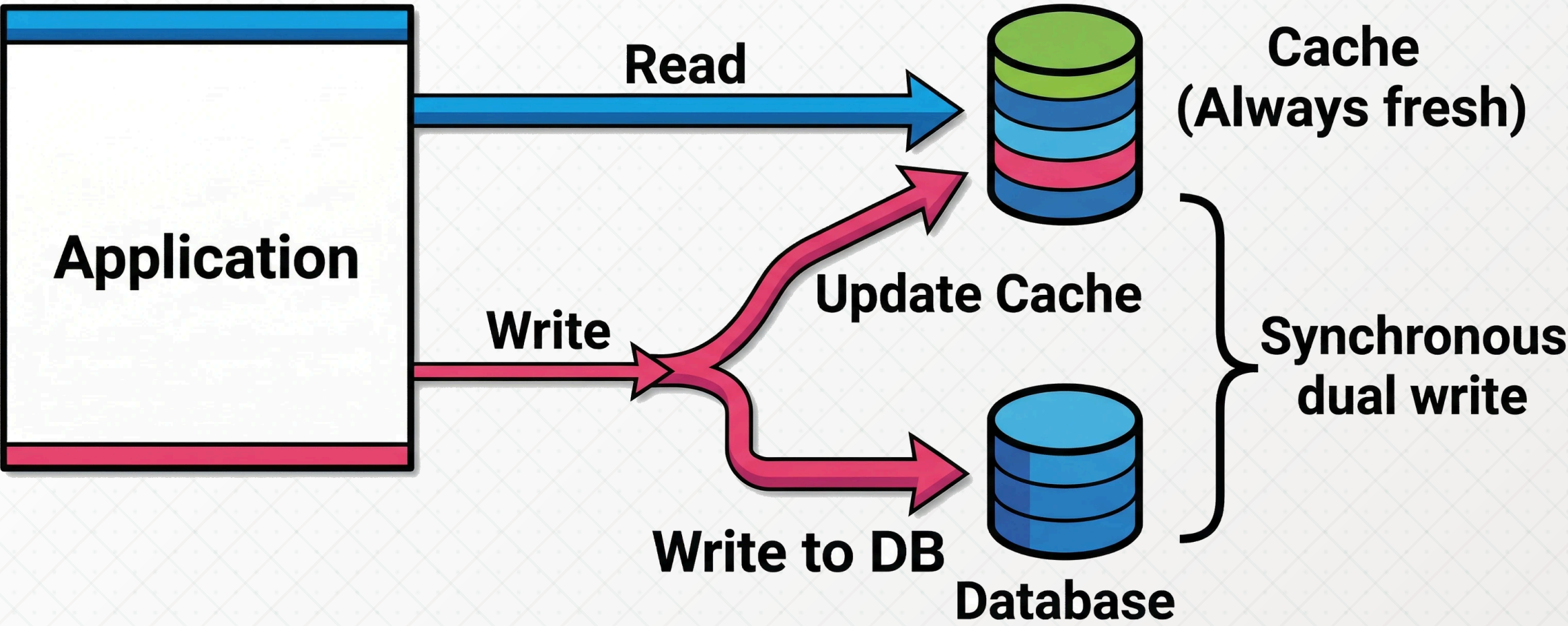
- ☕ **Đọc:** kiểm tra cache trước → cache hit trả về ngay; cache miss lấy từ DB rồi lưu cache
- ☕ **Ghi:** ghi vào database, sau đó **xóa cache** cũ để tránh dữ liệu cũ (stale data)
- ☕ Ứng dụng kiểm soát hoàn toàn khi nào đọc và ghi cache
- ☕ Nhược điểm: lần đầu tiên cache miss vẫn chậm; phải tự quản lý *invalidation*



Write-Through

Mỗi lần **ghi dữ liệu**, cập nhật đồng thời **cả database lẫn cache** — đảm bảo cache luôn mới nhất.

- ☕ Ghi X → ghi vào database **và** cập nhật cache ngay lập tức trong cùng một thao tác
- ☕ Cache luôn chứa dữ liệu mới nhất sau mỗi lần ghi
- ☕ Phù hợp: ứng dụng **đọc nhiều, ghi ít**, yêu cầu tính nhất quán cao
- ☕ Nhược điểm: mỗi lần ghi cần **hai thao tác** (DB + cache) → ghi chậm hơn

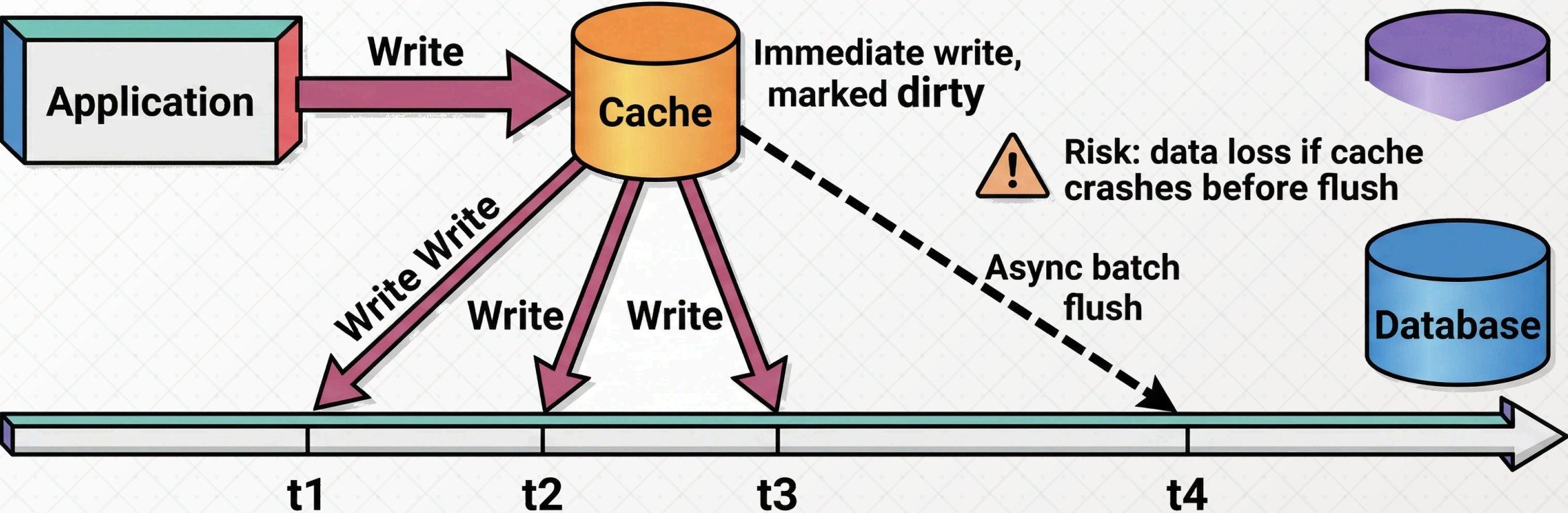


Write-Back (Write-Behind)

Ghi vào **cache trước**, ghi xuống database **sau** theo lịch trình — ưu tiên tốc độ ghi cao.

- Ghi X → lưu vào cache ngay, đánh dấu là *dirty*, database được cập nhật theo batch sau
- Ghi rất nhanh vì không chờ database tại thời điểm người dùng tương tác
- Nhiều lần cập nhật cùng một dữ liệu → chỉ cần ghi database một lần cuối
- **Rủi ro mất dữ liệu** nếu cache (RAM) bị lỗi trước khi kịp flush xuống database

Timeline - Write-Back pattern



Refresh-Ahead (Prefetching)

Chủ động **làm mới cache trước khi hết hạn** để người dùng không bao giờ gặp cache miss.

- ☕ Cache có TTL (thời gian sống); hệ thống tự làm mới *trước khi TTL hết*
- ☕ Ví dụ: cache hết hạn sau 10 phút → hệ thống tự fetch dữ liệu mới ở phút thứ 9
- ☕ Phù hợp: dữ liệu có thể dự đoán được nhu cầu — trang chủ báo lúc 7 giờ sáng cao điểm
- ☕ Nhược điểm: *lãng phí tài nguyên* nếu dự đoán sai — cập nhật dữ liệu không ai hỏi đến

Cache Eviction Policy là gì?

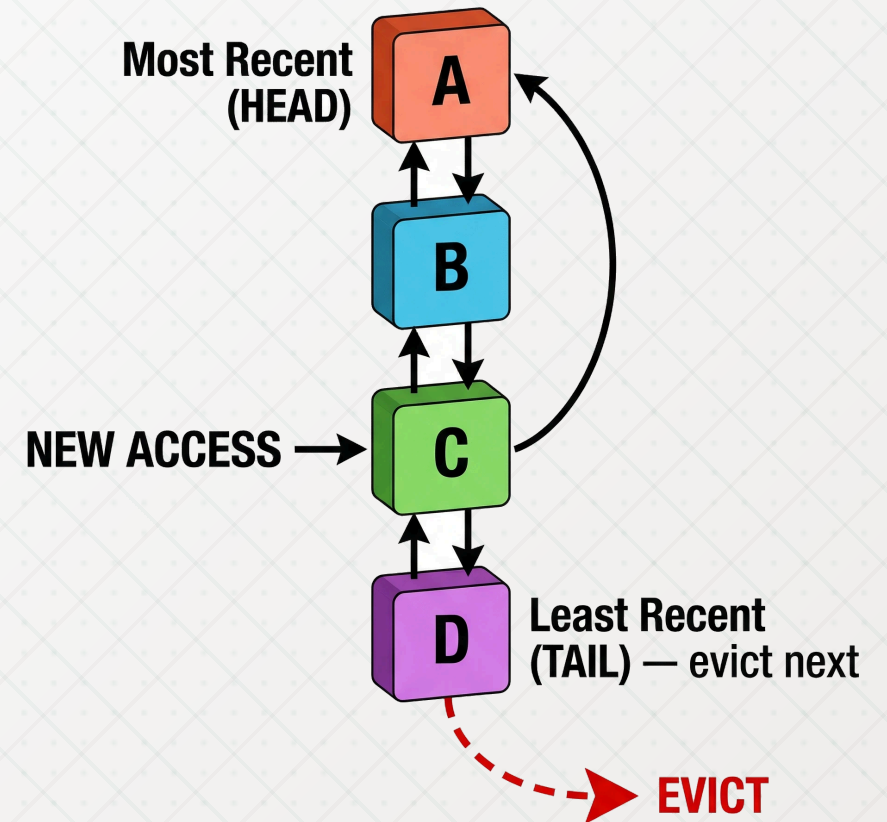
Cache Eviction Policy là quy tắc quyết định **dữ liệu nào bị xóa khỏi cache** khi cache đầy và cần chỗ cho dữ liệu mới.

- ☕ Cache có kích thước giới hạn — không thể lưu mọi thứ mãi mãi
- ☕ Khi cache đầy, hệ thống phải chọn *hy sinh* một mục để nhường chỗ
- ☕ Chọn sai mục để xóa → tỷ lệ cache miss tăng → hiệu suất giảm
- ☕ Hai chính sách phổ biến nhất: **LRU** (Least Recently Used) và **LFU** (Least Frequently Used)

LRU — Least Recently Used

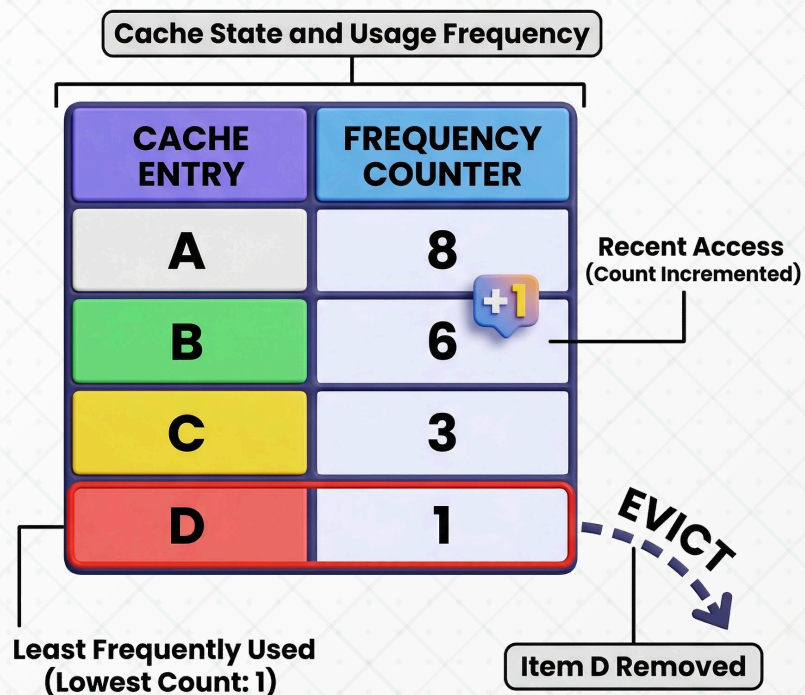
LRU xóa mục được truy cập **lâu nhất mà chưa dùng lại** — ưu tiên giữ lại những gì mới được sử dụng gần đây.

- Nguyên lý: dữ liệu vừa dùng ***có khả năng cao*** sẽ được dùng lại sớm
- Mỗi lần truy cập một mục → đưa mục đó lên **đầu danh sách ưu tiên**
- Khi cache đầy → xóa mục ở **cuối danh sách** (lâu nhất không được dùng)
- Triển khai điển hình: **HashMap + Doubly Linked List** —



LFU — Least Frequently Used

LFU CACHE EVICTION DIAGRAM



LFU xóa mục được truy cập **ít lần nhất** trong toàn bộ lịch sử — ưu tiên giữ lại những gì được hỏi đến nhiều nhất.

- Nguyên lý: dữ liệu được truy cập nhiều lần **có giá trị lâu dài** hơn dữ liệu chỉ mới dùng một lần
- Mỗi mục có **bộ đếm tần suất**; mỗi lần truy cập tăng bộ đếm lên 1
- Khi cache đầy → xóa mục có **bộ đếm thấp nhất** (nếu bằng nhau, dùng LRU làm tie-breaker)
- Nhược điểm: dữ liệu cũ được dùng nhiều trong quá khứ

LRU vs LFU — so sánh và lựa chọn

Không có chính sách nào tốt hơn tuyệt đối — chọn dựa trên **đặc điểm truy cập** của hệ thống.

LRU — phù hợp khi:

- ☕ Dữ liệu có tính *temporal locality* cao
- ☕ Người dùng thường xem lại nội dung vừa truy cập
- ☕ Workload thay đổi theo thời gian (trending content)
- ☕ Redis mặc định dùng xấp xỉ LRU

LFU — phù hợp khi:

- ☕ Có tập dữ liệu "hot" ổn định, ít thay đổi
- ☕ Tần suất truy cập phản ánh giá trị lâu dài
- ☕ Muốn tránh cache bị chiếm bởi dữ liệu chỉ dùng một lần
- ☕ Memcached hỗ trợ LFU tùy cấu hình

So sánh các chiến lược cache

Mỗi chiến lược có đánh đổi riêng — chọn dựa trên **đặc điểm đọc/ghi** của hệ thống.

Chiến lược	Điểm mạnh
Cache-Aside	Đơn giản, linh hoạt
Write-Through	Cache luôn mới nhất
Write-Back	Ghi cực nhanh
Refresh-Ahead	Không có cache miss

Chiến lược	Điểm yếu
Cache-Aside	Miss chậm lần đầu
Write-Through	Ghi chậm hơn
Write-Back	Rủi ro mất dữ liệu
Refresh-Ahead	Lãng phí nếu sai

Ví dụ thực tế: cache tỷ giá ngoại tệ

Cache hiệu quả nhất khi dữ liệu **thay đổi ít** nhưng được **đọc rất thường xuyên**.

- Tỷ giá USD cập nhật 2 lần/ngày nhưng hàng nghìn người hỏi mỗi phút
- Lần đầu tiên trong ngày: gọi API ngân hàng → lưu kết quả vào cache với TTL phù hợp
- Mọi request sau: phục vụ từ cache ngay lập tức, không gọi thêm API
- Hệ thống chịu được hàng nghìn người dùng dù ngân hàng chỉ được gọi 2 lần/ngày

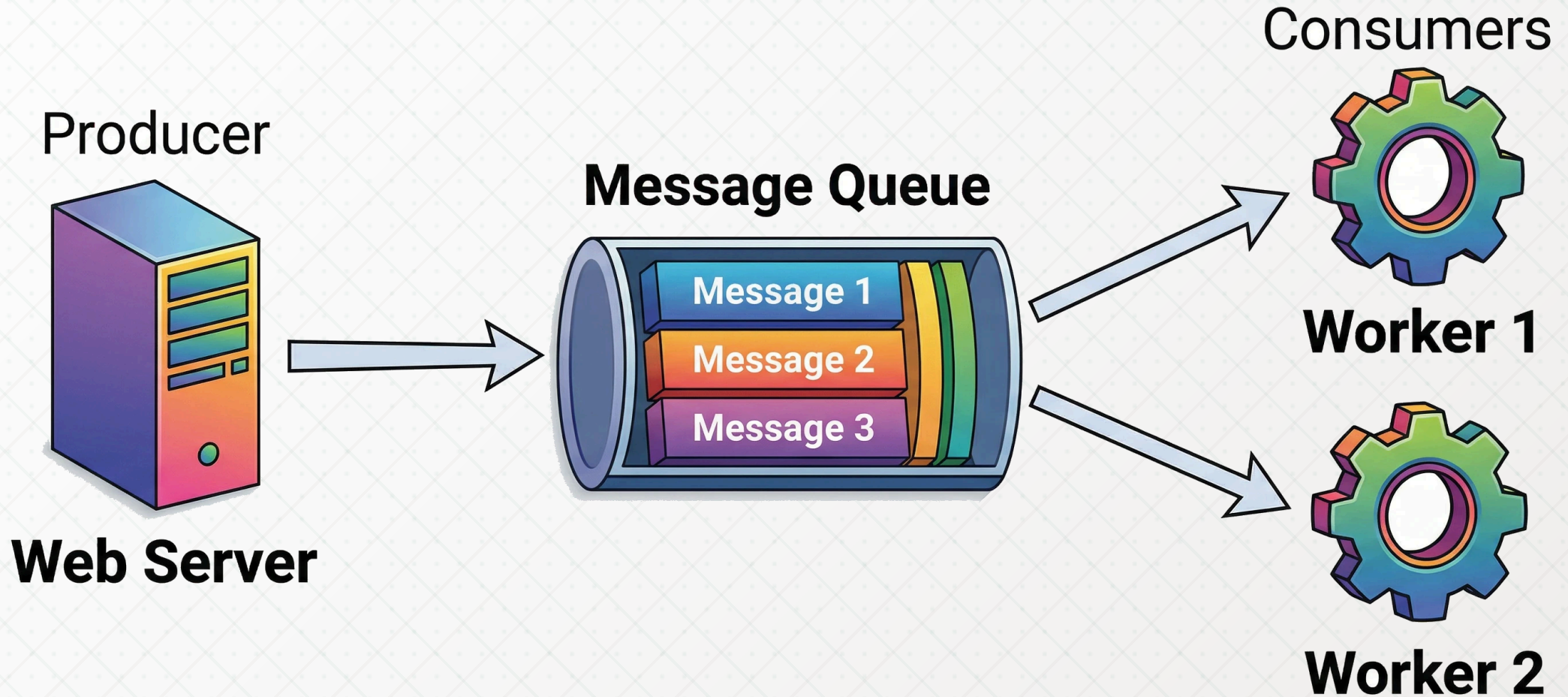
Hàng đợi thông điệp (Message Queue)

Message Queue là gì?

Message Queue là thành phần trung gian giữ các thông điệp, cho phép xử lý công việc **bất đồng bộ** — không cần kết quả ngay lập tức.

- ☕ **Producer** — gửi thông điệp (nhiệm vụ cần làm) vào hàng đợi rồi tiếp tục việc khác
- ☕ **Queue** — lưu trữ thông điệp cho đến khi có consumer nhận xử lý (nguyên tắc **FIFO**)
- ☕ **Consumer** — lấy thông điệp từ queue và thực hiện công việc tương ứng
- ☕ Producer và consumer ***không giao tiếp trực tiếp*** — hoàn toàn tách biệt qua queue

Minh họa: mô hình Producer - Queue - Consumer



Khi nào nên dùng Message Queue?

Dùng Message Queue khi cần **tách công việc xử lý sau** hoặc **điều hòa nhịp độ** giữa các thành phần hệ thống.

- ☕ **Background jobs** — gửi email xác nhận đăng ký, tạo báo cáo, gửi thông báo push
- ☕ **Giảm tải đỉnh điểm** — flash sale, livestream: queue làm bộ đệm tránh server bị sập đột ngột
- ☕ **Giao tiếp microservices** — service A gửi event, service B và C lắng nghe và tự xử lý
- ☕ **Tracking và logging** — thu thập sự kiện người dùng (click, view) xử lý hàng loạt sau

Các hệ thống Message Queue phổ biến

Biết tên các hệ thống cụ thể giúp câu trả lời phỏng vấn **thực tế và đáng tin cậy hơn**.

- ☕ **RabbitMQ** — message broker mã nguồn mở, giao thức AMQP, routing linh hoạt qua exchange
- ☕ **Apache Kafka** — nền tảng streaming phân tán, *throughput rất cao*, lưu message bền vững lâu dài
- ☕ **AWS SQS** — dịch vụ queue được AWS quản lý hoàn toàn, tự động mở rộng, không cần vận hành

Mô hình Producer - Queue - Consumer (chi tiết)

Tính **decoupling** là giá trị cốt lõi: các thành phần không cần biết nhau, chỉ giao tiếp qua queue.

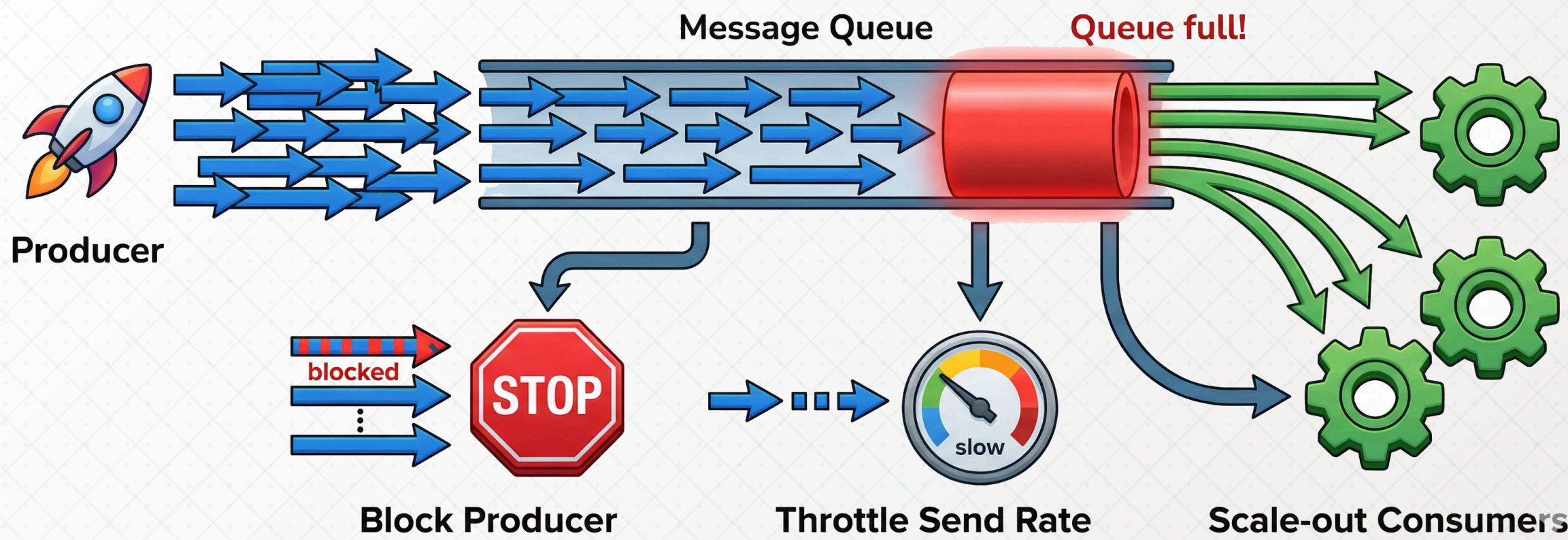
- ☕ **Producer** — tạo thông điệp và đẩy vào queue; không cần chờ consumer xử lý xong
- ☕ **Queue** — hoạt động theo **FIFO**; có thể lưu trong RAM (nhanh) hoặc trên đĩa (bền vững)
- ☕ **Consumer** — poll hoặc subscribe queue; nhiều consumer chạy song song để tăng tốc
- ☕ Muốn xử lý nhanh hơn: **tăng số consumer** mà không cần thay đổi bất kỳ gì ở producer

Cơ chế backpressure

Backpressure là cách hệ thống phản ứng khi queue đầy — tránh để producer gửi quá nhanh làm sập hệ thống.

- ☕ Vấn đề: producer gửi nhanh hơn consumer xử lý → message ùn ứ → hết bộ nhớ → sập
- ☕ **Chặn producer** — queue báo đầy, không nhận thêm message cho đến khi consumer xử lý bớt
- ☕ **Điều chỉnh tốc độ gửi** — service phát hiện queue dài → tự giảm tần suất gửi message
- ☕ **Scale-out consumer** — tự động thêm worker để xử lý nhanh hơn khi queue quá dài

UNDERSTANDING BACKPRESSURE IN A MESSAGE QUEUE SYSTEM

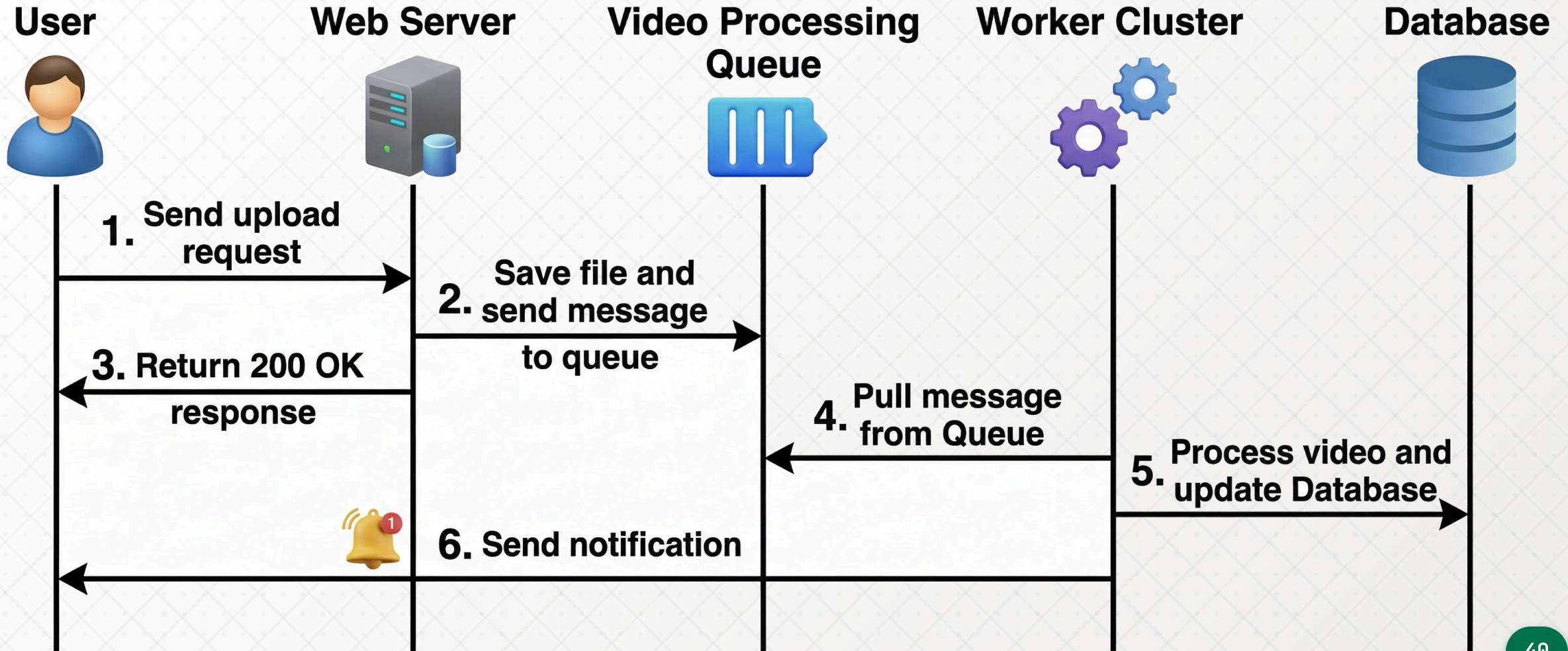


Ví dụ thực tế: xử lý video upload

Message Queue cho phép hệ thống **phản hồi người dùng ngay lập tức** trong khi xử lý tác vụ nặng ở hậu trường.

- ☕ Người dùng upload video → server lưu file và đẩy message "video 123, hãy chuyển đổi" vào queue
- ☕ Server phản hồi ngay: *"Video đang được xử lý, bạn sẽ nhận thông báo khi xong"*
- ☕ Worker nhận message → chuyển đổi định dạng video → cập nhật trạng thái → thông báo người dùng
- ☕ Nhiều video upload cùng lúc → xếp hàng trong queue → worker xử lý lần lượt hoặc song song

Minh họa: xử lý video với Message Queue



Bài tập thực hành

Bài tập 1: Thiết kế hệ thống đặt vé xem phim

Một nền tảng bán vé xem phim trực tuyến cần xử lý **lượng truy cập tăng đột biến** khi phim bom tấn mở bán.

Tình huống:

- ☕ 500,000 người dùng truy cập đồng thời khi mở bán vé phim bom tấn
- ☕ Xử lý thanh toán và gửi email xác nhận cho từng giao dịch
- ☕ Dữ liệu suất chiếu và giá vé ít thay đổi nhưng được đọc liên tục

Yêu cầu: Thiết kế kiến trúc có sử dụng *Load Balancer*, *Cache*, và *Message Queue*. Giải thích lý do chọn từng thành phần và cách chúng phối hợp với nhau.

Bài tập 2: Lựa chọn chiến lược cache

Cho các tình huống sau, hãy chọn **chiến lược cache phù hợp nhất** và giải thích lý do.

Tình huống	Chiến lược phù hợp
Hiển thị tỷ giá ngoại tệ, cập nhật 2 lần/ngày	?
Hệ thống game lưu điểm số hàng triệu lần/giây	?
Trang cấu hình hệ thống, cần đồng bộ ngay khi thay đổi	?
Trang chủ báo điện tử, cao điểm 7h sáng mỗi ngày	?

Thảo luận: Trường hợp nào *không nên* dùng cache? Tại sao?

Bài tập 3: Phân tích case study – Shopee Flash Sale

Mỗi đợt flash sale của **Shopee**, hàng triệu người dùng truy cập đồng thời để mua hàng giảm giá 90% trong vài phút đầu.

Phân tích các vấn đề kỹ thuật:

- ☕ Tại sao server Shopee không bị sập dù có hàng triệu request đồng thời?
- ☕ Thông tin sản phẩm, giá sale được hiển thị nhanh như thế nào?
- ☕ Đơn hàng được xử lý theo thứ tự nào và bằng cơ chế gì?

Thảo luận: Nếu bạn là kỹ sư thiết kế hệ thống flash sale này, bạn sẽ sử dụng kết hợp những kỹ thuật nào? Tại sao?

Bài tập 4: So sánh và lựa chọn Message Queue

Công ty khởi nghiệp của bạn đang xây dựng nền tảng thương mại điện tử và cần chọn giải pháp Message Queue.

Tình huống cụ thể:

-  **Nhóm A:** Startup nhỏ, 3 kỹ sư, ngân sách hạn chế, cần triển khai nhanh trên AWS
-  **Nhóm B:** Nền tảng analytics, thu thập 10 triệu sự kiện người dùng mỗi ngày, cần replay lại event
-  **Nhóm C:** Hệ thống đặt hàng, cần đảm bảo mỗi đơn hàng được xử lý đúng một lần

Yêu cầu: Mỗi nhóm chọn *một trong ba giải pháp* (AWS SQS, RabbitMQ, Apache Kafka) và trình bày lý do chọn cũng như các trade-off phải chấp nhận.

Q & A
